INWK6312 - Lab 2

Containers, Network Emulation, and Python Automation Foundations

Contents

Introduction

This lab covers Docker then use Containerlab to deploy a small mixed vendor topology, two Arista cEOS nodes and one Nokia SR Linux node, connected in a ring. You will bring up Layer 3 reachability on that topology by hand, using each vendor's own CLI. In the second part of the lab, you will build an isolated Python environment and use Netmiko to talk to the Arista nodes over SSH instead of typing commands yourself, parse the CLI output you get back, and handle a connection failure with a custom exception instead of letting the script crash.

Lab Objectives

By the end of this lab, you will be able to:

1. Inspect Docker images and containers to explain the difference between an image and a running container
2. Explain the components of a Containerlab topology definition file, including kinds, nodes, images, and links
3. Deploy a multi vendor topology combining Arista cEOS and Nokia SR Linux nodes
4. Manually configure Layer 3 interfaces on both Arista EOS and Nokia SR Linux, and confirm reachability across a fully meshed topology
5. Utilize the centralized Python virtual environment established in Lab 1 to manage project dependencies
6. Use Netmiko to connect to a network device over SSH and retrieve CLI output
7. Parse unstructured CLI output using a regular expression with named groups
8. Design and raise a custom exception class so a multi device script can report a connection failure without crashing

Lab Environment and Preparation

You will need:

- Your assigned Ubuntu VM IP address provided in Brightspace: _____.
- Docker, Containerlab, and Python 3 with the venv module preinstalled.
- The `ceos:v4.36` and `ghcr.io/nokia/srlinux:26.7.1-554-amd64` images already pulled and available locally.
- Your GitHub Classroom repository from Lab 1, cloned and up to date on this VM

If any of the components above are missing, check with your lab instructor before starting the lab.

Containerlab operations in this lab require sudo, since they create network namespaces and manage Docker on your behalf. Commands that need it are shown with sudo, if a command does not show sudo, you should not need it.

Part A: Containers and Network Emulation

Task 0: Docker Environment Review

Objective: confirm your environment is ready. You will not build or modify any images in this task.

1. Confirm Docker is available and check the version[1].

```
docker --version
```

2. List the images already on your VM.

```
docker image ls
```

   You should see `ceos:v4.36` and `ghcr.io/nokia/srlinux:26.7.1-554-amd64` in the list, alongside any other preloaded images.

3. Inspect the metadata of each image this lab will use. An image is an immutable snapshot, this pulls out just its creation date and size rather than the entire JSON blob.

```
docker image inspect ceos:v4.36 --format 'Created: {{.Created}}\nSize:
↪  {{.Size}} bytes'
docker image inspect ghcr.io/nokia/srlinux:26.7.1-554-amd64 --format
↪  'Created: {{.Created}}\nSize: {{.Size}} bytes'
```

4. Review Docker's own storage summary, this reflects the layered, shared storage model (covered on Lecture 4).

```
docker system df
```

5. Confirm Containerlab is installed and check its version[2].

```
containerlab version
```

Questions and Deliverables

1. Compare the Size field for the cEOS and the SR Linux images from step 3. Which one is larger, and does that match the DISK USAGE or CONTENT SIZE columns you would see in a full docker image ls output?
2. Why does an image stay the same, unchanged object no matter how many containers you create from it?

Task 1: Your First Containerlab Node, Arista cEOS

Objective: deploy a single cEOS node on its own, connect to its CLI, and destroy it, before creating any topology.

> Note: Both the cEOS and SR Linux images ship with a default login account already configured, and Containerlab automatically registers each deployed node's hostname

[1] This lab was tested using Docker v29.7.1.
[2] this lab was testing using Containerlab v0.77

in your VM's own `/etc/hosts` file, so you can reach a node by name instead of its management IP address.

1. Create the folder structure this lab will use.

```
mkdir -p ~/labs/lab2/topology ~/labs/lab2/scripts
cd ~/labs/lab2/topology
```

2. Create a small temporary topology file with a single cEOS node.

```
nano test-ceos.clab.yml
```

```
name: test-ceos
topology:
  nodes:
    ceos1:
      kind: arista_ceos
      image: ceos:v4.36
```

3. Deploy it.

```
sudo containerlab deploy -t test-ceos.clab.yml
```

4. Verify it came up and note its management IP address.

```
sudo containerlab inspect -t test-ceos.clab.yml
```

5. Connect directly to its CLI. Containerlab names the underlying container `clab-<lab-name>-<node-name>`, so this node is `clab-test-ceos-ceos1`.

```
docker exec -it clab-test-ceos-ceos1 Cli
```

   Log in via SSH using the default username/password=`admin/admin`.

```
ssh admin@clab-test-ceos-ceos1
```

6. From the EOS prompt, run two read only commands, then leave the CLI.

```
show version
show interfaces status
exit
```

7. Destroy the lab, you are done with this topology.

```
sudo containerlab destroy -t test-ceos.clab.yml --cleanup
```

The local `--cleanup` flag instructs containerlab to remove the auto-generated lab directory `clab-<lab-name>` and all its content. This prevents Containerlab from reusing previous startup configuration artifacts on the next deploy. In this case, you will not need any saved information.

Questions and Deliverables

1. Provide the output of containerlab inspect from step 4.

2. What EOS software version did show version report inside your container?
3. Which interface did the `show interfaces` command list?

Task 2: Adding Nokia SR Linux

Objective: deploy a single SR Linux node on its own, and get a first look at how its command structure differs from EOS, before destroying it.

1. Create a second temporary topology file.

```
cd ~/labs/lab2/topology
nano test-srl.clab.yml
```

```
name: test-srl
topology:
  nodes:
    srl1:
      kind: nokia_srlinux
      image: ghcr.io/nokia/srlinux:26.7.1-554-amd64
      type: ixr-d1
```

2. Deploy it and inspect it.

```
sudo containerlab deploy -t test-srl.clab.yml
sudo containerlab inspect -t test-srl.clab.yml
```

3. Connect to the SR Linux CLI. Note the command is `sr_cli`, not `Cli`. Type `quit` and `ENTER` to exit.

```
docker exec -it clab-test-srl-srl1 sr_cli
```

   Log in via SSH using default the username/password=`admin/NokiaSrl1!`.

```
ssh admin@clab-test-srl-srl1
```

4. Run two read only commands, then leave the CLI.

```
show version
show interface brief
quit
```

5. Destroy the lab.

```
sudo containerlab destroy -t test-srl.clab.yml --cleanup
```

Questions and Deliverables

1. Provide the output of show version from the SR Linux CLI.
2. The SR Linux prompt looks like `--{ running }--[ ]--`, while the EOS prompt you saw in Task 1 was a plain hostname and symbol. What does the running portion of the SR Linux prompt tell you that the EOS prompt does not?
3. Which interface is up?

Task 3: Building the Three Node Ring Topology

Objective: combine both kinds into the persistent topology that carries forward into the rest of this course.

Three nodes connected in a ring means every node has a direct link to both of the other two, a ring of three is the same thing as a full mesh. This matters for what comes next, none of these nodes will need to forward traffic on behalf of another, unlike the router you built by hand out of namespaces in Lab 1.

1. Create the real topology file.

```
cd ~/labs/lab2/topology
nano lab-net.clab.yml
```

```
name: lab-net
prefix: "" # Removes a prefix from the container names
topology:
  nodes:
    ceos1:
      kind: arista_ceos
      image: ceos:v4.36
    ceos2:
      kind: arista_ceos
      image: ceos:v4.36
    srl1:
      kind: nokia_srlinux
      image: ghcr.io/nokia/srlinux:26.7.1-554-amd64
      type: ixr-d1
  links:
    - endpoints: ["ceos1:eth1", "ceos2:eth1"]
    - endpoints: ["ceos2:eth2", "srl1:ethernet-1/1"]
    - endpoints: ["srl1:ethernet-1/2", "ceos1:eth2"]
```

2. Deploy it.

```
sudo containerlab deploy -t lab-net.clab.yml
```

3. Verify all three nodes are running.

```
sudo containerlab inspect -t lab-net.clab.yml
```

4. Confirm Containerlab also registered each node's hostname on your VM. Because the topology file sets prefix to an empty string, the hostname it registers is the plain node name, not a clab prefixed version of it.

```
cat /etc/hosts
```

   You should see a block bounded by CLAB-lab-net-START and CLAB-lab-net-END, mapping `ceos1`, `ceos2`, and `srl1` to their management IP addresses. From this point on, you can reach any of the three nodes by its name without needing to look up or type its management IP

address.

5. Cross check from the Docker side as well.

```
docker ps
```

6. Inspect the topology using the graph command, press `CTRL-C` to exit.

```
containerlab graph -t lab-net.clab.yml
```

Questions and Deliverables

1. Provide the output of containerlab `inspect -t lab-net.clab.yml`.
2. What does specifying a kind actually control in containerlab toplogy file?
3. Explain why the node names do not include the lab name prefix.

Task 4: Configuring Layer 3 Reachability

Objective: bring up an IP address on each side of every link by hand, using each vendor's own CLI.

Addressing plan for this topology:

Link	Node : Interface	IP Address
ceos1 ↔ ceos2	ceos1 : Ethernet1	10.0.12.1/30
ceos1 ↔ srl1	ceos1 : Ethernet2	10.0.13.2/30
ceos1 ↔ ceos2	ceos2 : Ethernet1	10.0.12.2/30
ceos2 ↔ srl1	ceos2 : Ethernet2	10.0.23.1/30
ceos2 ↔ srl1	srl1 : ethernet-1/1	10.0.23.2/30
ceos1 ↔ srl1	srl1 : ethernet-1/2	10.0.13.1/30

Configure each node fully, and verify it, before moving to the next one. A typo in an address here looks like a broken link in Task 5, and is much harder to trace back once you are three nodes in.

1. Connect to ceos1.

```
ssh admin@ceos1
```

2. Configure both of its data interfaces. On Arista EOS, a physical interface starts in switchport mode and must be explicitly converted to a routed port with no switchport before it will accept an IP address.

```
enable
configure
interface Ethernet1
   no switchport
   ip address 10.0.12.1/30
   no shutdown
interface Ethernet2
   no switchport
   ip address 10.0.13.2/30
```

```
   no shutdown
end
```

3. Verify before moving on.

```
show ip interface brief
```

Both Ethernet1 and Ethernet2 should show your assigned addresses with a status and protocol of up. Then leave the CLI with `exit`.

4. Repeat the same pattern on ceos2, with its own addresses.

```
ssh admin@ceos2
```

```
enable
configure
interface Ethernet1
   no switchport
   ip address 10.0.12.2/30
   no shutdown
interface Ethernet2
   no switchport
   ip address 10.0.23.1/30
   no shutdown
end
show ip interface brief
```

5. Connect to srl1. SR Linux configuration happens inside a candidate datastore, which you commit explicitly, and a subinterface only forwards traffic once it is associated with a network instance.

```
ssh admin@srl1
```

```
enter candidate
set / interface ethernet-1/1 subinterface 0 ipv4 admin-state enable
set / interface ethernet-1/1 subinterface 0 ipv4 address 10.0.23.2/30
set / interface ethernet-1/2 subinterface 0 ipv4 admin-state enable
set / interface ethernet-1/2 subinterface 0 ipv4 address 10.0.13.1/30
set / network-instance default interface ethernet-1/1.0
set / network-instance default interface ethernet-1/2.0
```

6. Verify before moving on.

```
diff
```

The `admin-state` of both `ethernet-1/1.0` and `ethernet-1/2.0` should show as `enable`, with the addresses you just assigned. Also both interfaces should be assigned to the `default` network instance.

7. Commit and exit

```
commit now
quit
```

If commit is not successful, enter `discard stay` and repeat steps 5 to 7.

Questions and Deliverables

1. Provide the `show ip interface brief` output from `ceos1` and `ceos2`.
2. Provide the output of the `diff` command from srl1.

Task 5: Verifying the Full Mesh

Objective: confirm every node can reach both of its neighbors directly. Use Docker command or log into the node and use the CLI.

1. From `ceos1`, ping both neighbors (without logging-in).

```
docker exec -it ceos1 Cli -c "ping 10.0.12.2"
docker exec -it ceos1 Cli -c "ping 10.0.13.1"
```

2. From `ceos2`, ping both neighbors.

```
docker exec -it ceos2 Cli -c "ping 10.0.12.1"
docker exec -it ceos2 Cli -c "ping 10.0.23.2"
```

3. From srl1, ping both neighbors. SR Linux requires the network instance to be specified on the ping command itself.

```
docker exec -it srl1 sr_cli ping -c 5 10.0.23.1 network-instance default
docker exec -it srl1 sr_cli ping -c 5 10.0.13.2 network-instance default
```

Note: Use `CTRL-c` multiple times then `CTRL-d` or `CTRL-q` to force quit.

Questions and Deliverables

1. Provide the output of all six ping tests.

Part B: Python Foundations and Netmiko

Task 6: Building an Isolated Python Environment

Objective: Activate the virtual environment and update the global dependencies.

1. Move into your labs folder.

```
cd ~/labs
```

2. Activate the virtual environment created in Lab 1.

```
source .velab/bin/activate
```

   Your prompt should now be prefixed with (.velab).

3. Install Netmiko inside the environment.

```
pip install netmiko
```

4. Update the cumulative requirements file at the root of your repository.

```
pip freeze > requirements.txt
```

Questions and Deliverables

1. Provide the updated contents of `~/labs/requirements.txt`.
2. If a classmate cloned your repository onto their own VM and created a fresh virtual environment, what single pip command would recreate the entire course environment from the root `requirements.txt`?

Task 7: Your First Netmiko Connection

Objective: connect to ceos1 over SSH using Netmiko instead of docker exec, and pull the output of one command.

1. Make sure your virtual environment from Task 6 is still active, then create a script in `~/labs/lab2/scripts`.

```
cd ~/labs/lab2
nano scripts/connect_ceos1.py
```

```
from netmiko import ConnectHandler

ceos1 = {
    "device_type": "arista_eos",
    "host": "ceos1",
    "username": "admin",
    "password": "admin",
}

connection = ConnectHandler(**ceos1)
```

```
output = connection.send_command("show ip interface brief")
print(output)
connection.disconnect()
```

This code uses the node's name `ceos1`. You can also use the node's management IP address, if you prefer.

2. Run it.

```
python3 scripts/connect_ceos1.py
```

3. Verify the printed output matches what you saw directly on the CLI in Task 4, Netmiko is automating the same SSH session you could type by hand, it should show the same two interfaces and addresses.

Questions and Deliverables

1. Provide the output of running this script.
2. The dictionary sets `device_type` to `arista_eos`. What does Netmiko use this value for internally?

Task 8: Parsing CLI Output with Regular Expressions

Objective: turn the raw text from Task 7 into structured data your own code can use, instead of text a human has to read.

1. Create a new script that parses the output using a regex with named groups, the same pattern used in Lecture 3.

```
nano scripts/parse_ceos1.py
```

```
import re
from netmiko import ConnectHandler

ceos1 = {
    "device_type": "arista_eos",
    "host": "ceos1",
    "username": "admin",
    "password": "admin",
}

connection = ConnectHandler(**ceos1)
output = connection.send_command("show ip interface brief")
connection.disconnect()

pattern =
↪  r"(?P<interface>\S+)\s+(?P<ip>\d+\.\d+\.\d+\.\d+/\d+)\s+(?P<status>\S+)\s+(?P<protocol

for match in re.finditer(pattern, output):
    print(match.group("interface"), match.group("ip"),
    ↪  match.group("status"))
```

2. Run it, and confirm it prints one line per interface that has an interface name, IP address, and status, correctly split into separate fields.

```
python3 scripts/parse_ceos1.py
```

3. Add an if statement so the script only prints non-management interfaces whose status is up, without a change to the regex itself.

Questions and Deliverables

1. Provide the output of the script from step 2.
2. Provide your modified code from step 3, and its output.

Task 9: Handling Failures with a Custom Exception

Objective: extend the script to loop over both cEOS nodes, and make sure one unreachable device does not stop the script from checking the other.

1. Create the script.

```
nano scripts/inventory_check.py
```

```
from netmiko import ConnectHandler, NetmikoTimeoutException,
↪  NetmikoAuthenticationException


class DeviceConnectionError(Exception):
    """Raised when Netmiko cannot reach or authenticate to a device."""
    pass


devices = [
    {"name": "ceos1", "device_type": "arista_eos", "host": "ceos1",
↪  "username": "admin", "password": "admin"},
    {"name": "ceos2", "device_type": "arista_eos", "host": "ceos2",
↪  "username": "admin", "password": "admin"},
]

for device in devices:
    params = {key: value for key, value in device.items() if key != "name"}
    try:
        try:
            connection = ConnectHandler(**params)
        except (NetmikoTimeoutException, NetmikoAuthenticationException) as
        ↪  exc:
            raise DeviceConnectionError(f"Could not reach {device['name']}:
            ↪  {exc}") from exc
        output = connection.send_command("show ip interface brief")
```

```
        print(f"--- {device['name']} ---")
        print(output)
        connection.disconnect()
    except DeviceConnectionError as exc:
        print(f"Skipping {device['name']}: {exc}")
        continue
```

2. Run it, and confirm both devices report correctly.

```
python3 scripts/inventory_check.py
```

3. Now deliberately break it. Change the host address for node `ceos1` to something else unreachable, such as `host=ceos3` or `host=10.99.0.1` and run the script again.
4. Confirm the script prints a clear Skipping message for ceos1, naming it specifically, and still completes successfully for ceos2, rather than stopping on the first failure.
5. Restore `ceos1`'s correct address before moving on.

Questions and Deliverables

1. Provide the output from both the working run in step 2 and the deliberately broken run in step 3.
2. *NetmikoTimeoutException* and *NetmikoAuthenticationException* are caught specifically here, rather than a single bare except. Give one practical reason a script managing many devices would want to know which of these two happened, rather than only knowing that something failed.

Task 10: Committing Your Work

Objective: Stage your changes using the global Git configuration established in Lab 1.

1. Move to the root of your repository.

```
cd ~/labs
git status
```

2. Stage your `lab2` directory and the updated `requirements.txt`.

```
git add lab2 requirements.txt .gitignore
git status
```

3. Confirm that no environment folders or temporary `Containerlab` files are being tracked. `git add` command is likely to produce an error about adding embedded git repository if `**/clab-*/` is not included in `.gitignore`.
4. Commit and push.

```
git commit -m "Add Lab 2 ring topology and Netmiko automation scripts"
git push
```

Questions and Deliverables

1. Provide the output of `git status`, confirming the global `.gitignore` is protecting the repository from "dirty" commits.
2. Provide the output of `git log --oneline -5` after your commit.

Clean Up

You can destroy the lab now, but before that, you will need to save the device configurations to be reused in future labs. Typically, you would do that in each device individually using commands such as `write memory`, but containerlab offers a convenient way to to perform configuration save for all containers running in the lab.

```
sudo containerlab save -t ~/labs/lab2/topology/lab-net.clab.yml
```

The `save` command will save the configuration files under the directory `lab2/topology/clab-lab-net`, which is not tracked by git.

Destroy the topology:

```
sudo containerlab destroy -t ~/labs/lab2/topology/lab-net.clab.yml
```

Destroying and later redeploying this topology will bring the containers back and restore the topology with the saved configurations.

Deactivate the virtual environment:

```
deactivate
```

Submission

Confirm your repository includes, at minimum, the following, then submit as instructed by your course delivery platform:

- labs/lab2/topology/lab-net.clab.yml
- labs/lab2/scripts/connect_ceos1.py, parse_ceos1.py, and inventory_check.py
- The updated root level `~/labs/requirements.txt`
- Your answers to the Questions and Deliverables sections, submitted as your lab report per your instructor's separate instructions
- git log `git log --oneline --graph -10`

Appendix: Command Summary

Docker and Containerlab

Command	Usage
docker version	Confirm the installed Docker version
docker images	List Docker images available on the host
docker system df	Show Docker disk usage and storage summary
containerlab version	Confirm the installed Containerlab version
sudo containerlab deploy -t <topology file>	Deploy the multi vendor topology
containerlab inspect -t <topology file>	View the status and management IP addresses of nodes
sudo containerlab graph -t <topology file>	Generate a visual graph of the topology
sudo containerlab save -t <topology file>	Save the running configuration of all nodes
sudo containerlab destroy -t <topology file>	Stop the lab and remove containers
ssh admin@ceos1	Connect to a node CLI via SSH

Arista EOS CLI Essentials

Command	Usage
enable	Enter privileged EXEC mode
configure	Enter global configuration mode
no switchport	Convert a layer 2 interface into a routed layer 3 port
show ip interface brief	Display the status and IP addresses of interfaces
show version	Report the EOS software version

Nokia SR Linux CLI Essentials

Command	Usage
sr_cli	Enter the SR Linux interactive CLI
enter candidate	Enter the candidate datastore for configuration
commit save	Commit changes and save them to the startup configuration
discard stay	Discard uncommitted changes while remaining in the CLI
show interface	Display interface status and addressing

Python and Virtual Environments

Command	Usage
source ~/labs/.velab/bin/activate	Activate the centralized virtual environment
pip install netmiko	Install the Netmiko library for SSH automation
pip freeze > ~/labs/requirements.txt	Update the cumulative requirements file
python3 connect_ceos1.py	Execute a Python automation script
deactivate	Exit the virtual environment

Command	Usage

Git Version Control

Command	Usage
git status	Check the status of staged and unstaged changes
git add <directory/>	Stage the directory for commit
git commit -m “message”	Record staged changes to the repository history
git push	Upload local commits to the remote GitHub repository
git log –oneline -5	View a simplified history of the last five commits

Default Credentials

Kind	Username	Password
Arista cEOS	admin	admin
Nokia SR Linux	admin	NokiaSrl1!